camelAI

← ALL POSTS

ENGINEERING · OCT 6, 2026 · 12 MIN READ

Agent Harness vs Framework vs Runtime vs Sandbox, Explained



Illiana Reed
CEO & Product, camelAI



TL;DR: A framework is a library for building your own agent loop (LangGraph, CrewAI, Mastra). A harness is a loop someone already built, with the system prompt, tools and context management included (Claude Code, Codex, Pi, Deep Agents), so the two fill the same layer. A runtime is where agents live over time: it keeps their state, wakes them, scales them and recovers them (camelRun, AgentCore Runtime, Cloudflare Agents). A sandbox isolates the code the model writes (E2B, Modal, Vercel Sandbox).

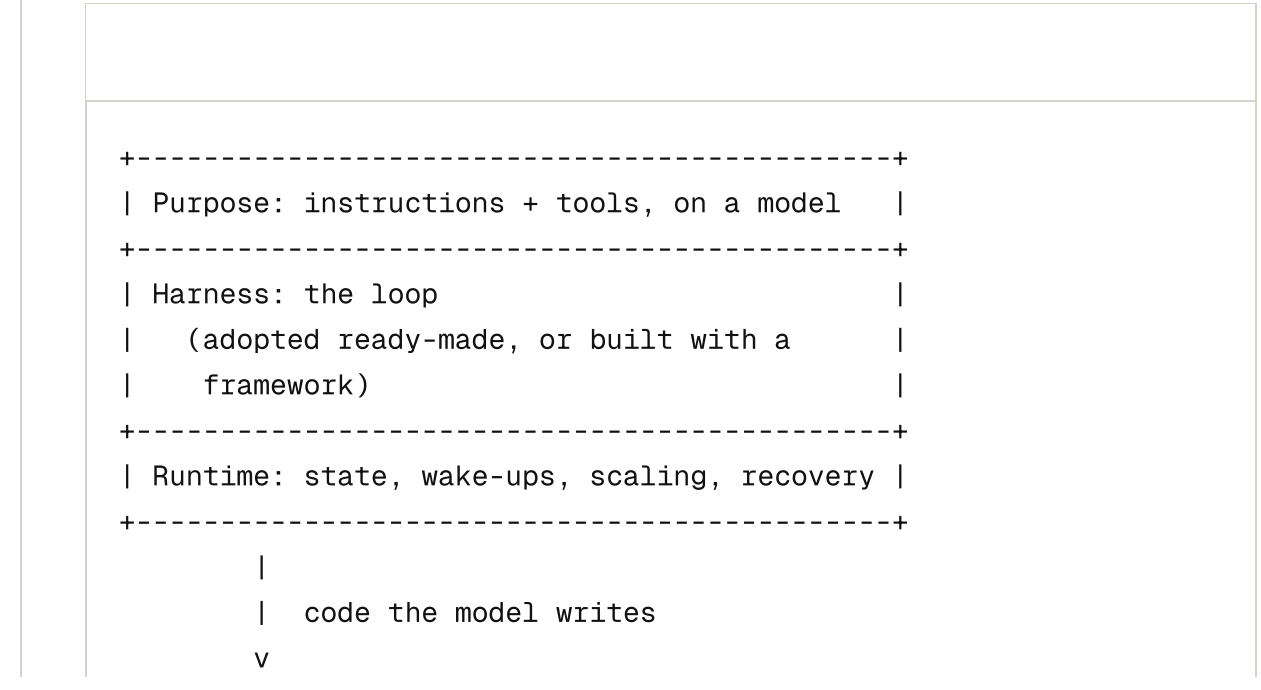
Agent harness vs framework is the question teams tend to hit once the demo works, and the vocabulary doesn't help. LangGraph calls itself an orchestration framework, while the Deep Agents README calls it a graph runtime. E2B calls its sandbox backend a runtime too. And Anthropic, OpenAI and AWS now all ship products they call a harness.

This post defines four layers by what they do, not by what they're called. It gives a

boundary test for each pair and shows where your own prompt and tools sit. For build choices, see how to build an AI agent. For products, see AI agent runtimes and hosting platforms compared.

The four layers at a glance

Layer	What it does	Examples	You need it when...
Framework	Gives you parts to build a loop: model adapters, tool schemas, state, graphs	LangGraph, CrewAI, Mastra, Pydantic AI, OpenAI Agents SDK, Google ADK	You want to own the control flow
Harness	Runs a ready-made loop: calls the model, runs tools, manages and compacts context	Claude Agent SDK / Claude Code, Codex, Pi, OpenCode, Deep Agents, Strands harness	You want proven agent behavior without designing the loop
Runtime	Keeps agents alive between turns: durable state, identity, wake-ups, scaling, recovery	camelRun, AgentCore Runtime, Cloudflare Agents, LangSmith Deployment	Agents outlive a request: many users, long waits, schedules, deploys
Sandbox	Isolates model-written code: filesystem, network, CPU and memory limits	E2B, Modal Sandboxes, Vercel Sandbox, Firecracker, gVisor	The model writes or runs code nobody reviewed



```
+--------------------------------+
| Sandbox: isolated execution    |
+--------------------------------+
```

A framework isn't a layer of its own; it's one way to produce the harness layer. The sandbox hangs off to the side because it exists for the model's code, not for yours.

Framework: parts for building a loop

A framework is a library of building blocks for writing an agent loop: a model client that hides provider differences, tool declarations, state and memory, and often a graph or workflow model for control flow. LangGraph, CrewAI, Mastra, Pydantic AI, the OpenAI Agents SDK and Google ADK are frameworks.

A framework usually leaves more decisions about the work to you. It may supply a ready-made loop, but you choose the instructions, tools and context-management policy, or adopt a more opinionated harness package. A library also needs a place to run. LangGraph's durable execution checkpoints a graph so a run can resume after a failure. You still operate the process and the checkpoint store yourself, or hand both to LangSmith Deployment.

Harness: a loop that's already built

A harness is a finished, opinionated agent loop. It calls the model, runs the tool calls the model returns, feeds the results back and repeats until the task is done. Depending on the harness, it also ships some of these:

- a system prompt the vendor has tuned;
- built-in tools for files, commands and web search;
- context management, including compaction;
- permissions, subagents and resumable sessions.

Anthropic describes the Claude Agent SDK as Claude Code's tools, agent loop and context management, packaged as a library. Codex, Pi, OpenCode, Deep Agents and the Strands harness are the same kind of thing.

The big vendors now use the word officially:

- **OpenAI.** The Agents API, in public beta since September 10, 2026, gives your

app what OpenAI calls the managed Codex harness. OpenAI also publishes the Codex harness as open source.

- **Strands.** The Strands Agents SDK repository is now harness-sdk. Its `create_harness()` returns an assembled agent with shell, file and web tools, memory and context management.
- **AWS.** AgentCore harness went GA on June 17, 2026. It runs a loop you define in configuration (model, tools, skills, instructions) and can export it to Strands code.
- **Anthropic.** Its docs call Claude Managed Agents a pre-built, configurable agent harness.

Agent harness vs framework: who assembled the loop

Framework and harness aren't stacked. They compete for the same slot: the loop that calls your model. The difference is who made the decisions inside it.

The test: give it a task and a model key, and write nothing else. If it does useful work without you writing a loop, tools or a prompt, it's a harness. If you have to assemble those pieces before anything runs, it's a framework.

The line blurs from both directions:

- **Harnesses are often built on frameworks.** The Deep Agents README describes three layers: LangGraph at the bottom, LangChain's `create_agent` as a minimal harness on top, and Deep Agents as a more opinionated harness above that.
- **Frameworks now ship harnesses.** Pydantic AI's docs say every Pydantic AI agent already has a light harness. A separate Harness package adds compaction, subagents, memory and sandboxes.

Choose a framework when the shape of the work is yours: a workflow with LLM steps, a routing graph, or a loop you want to evaluate step by step. Choose a harness for open-ended tasks with files and tools, where the vendor has already tuned the prompts, tools and compaction for you.

Runtime: where the agent lives between turns

A harness runs a turn; a runtime runs an agent's whole life. A runtime is the service that keeps an agent addressable and alive for days or months:

- **State:** history, files and pending work, stored outside any one process

- **State:** history, files and pending work, stored outside any one process.
- **Identity:** a stable id, so a user, ticket or thread gets the same agent every time.
- **Wake-ups:** messages, schedules, webhooks, channel events and approvals start work without any process of yours waiting.
- **Scaling:** most agents stay asleep while a few are awake, each loaded wherever there's room.
- **Recovery:** recorded progress supports resuming an interrupted turn or reporting where it stopped; the guarantees vary by runtime. Exactly-once external side effects still need idempotent tools.

Runtimes make different trade-offs:

- AgentCore Runtime gives each session a microVM for up to 8 hours per lifecycle (14 days on its Instances compute type). Compute state is ephemeral, so you add session storage or AgentCore Memory to keep anything.
- Each Cloudflare agent is a Durable Object with SQL-backed state and a scheduler, and it hibernates when idle.
- LangSmith Deployment hosts agents from any framework, with checkpoints, cron jobs and human-in-the-loop.
- camelRun keys agents by your own ids and keeps their history and files until you delete them.

Anthropic's guide to hosting the Agent SDK shows why this layer exists. Each session is a CLI subprocess that owns a shell, a working directory and transcripts on local disk. By default, none of it survives a container restart, a scale-down or a move to another node, so the guide goes on to session stores, container patterns and session routing. That work is a runtime, whether a vendor provides it or you build it.

Libraries also use "runtime" for the in-process engine that executes a loop, as in the Deep Agents description of LangGraph. In this post, a runtime is the service an agent lives in.

Agent harness vs runtime: one turn vs a lifetime

The test: kill the process mid-turn and wait a day. Does the agent come back with its history and files? Can a schedule wake it tomorrow with no process of yours running? If yes to both, something is acting as a runtime. If not, you have a harness running in a process.

The same vendors now offer their harness both self-run and hosted, which shows these really are separate layers:

Vendor	Run the harness yourself	Use it hosted
OpenAI	Open-source Codex CLI or SDK in your container	Agents API, a managed Codex harness
Anthropic	Claude Agent SDK in your container	Claude Managed Agents, a hosted harness
AWS / Strands	Strands harness in your process	AgentCore harness, which exports to Strands code

Durable execution engines such as Temporal, Inngest and Restate supply one runtime property: surviving crashes mid-workflow, for a loop you write as a workflow. Durable AI agents covers how they differ from agent runtimes.

AgentCore harness vs runtime

AWS ships both layers under the AgentCore name, so the two get mixed up. AgentCore Runtime hosts a loop you build, in any framework or none. You package the agent as a container, and each session gets its own microVM for up to 8 hours (14 days on Instances). AgentCore harness, generally available since June 17, 2026, is the loop itself. You declare the model, system prompt, tools, skills and memory, and AWS runs the loop on Runtime, so every harness session is a Runtime microVM too.

The difference shows up in state. A Runtime session's compute state is ephemeral, so to keep anything past a session you add AgentCore Memory or session storage yourself. A harness created through the API provisions managed memory by default, so its conversations continue across sessions. Pick Runtime when you want to own the loop, and harness when configuration is enough; harness can export to Strands code if you outgrow it. The harness has no separate charge: you pay for the Runtime, Memory and tools it uses. AWS Bedrock AgentCore pricing explained breaks down each component and compares it with camelRun.

Sandbox: where the model's code runs

A sandbox is an isolated place to run code nobody reviewed, such as the commands and scripts a model writes. It limits what that code can reach: the

commands and scripts a model writes. It limits what that code can reach: the host, other tenants' data, and whatever secrets or network destinations your policy excludes. It also limits CPU, memory and time.

Isolation ranges from heavy to light:

- **MicroVMs** give each sandbox its own kernel. E2B and Vercel Sandbox run Firecracker microVMs, and AgentCore Runtime gives each session a microVM.
- **User-space kernels** such as gVisor intercept system calls. Modal Sandboxes use gVisor by default and offer a VM option.
- **Language-level sandboxes** run one language and expose no operating system to the code. Cloudflare's Code Mode runs model-written code in V8 isolates that start in milliseconds.

camelRun sits at the light end. The model writes JavaScript that calls the agent's tools and reads its files, and that's all the code can do. There's no shell, no Python, no package installs and no network. (camelRun's Python SDK is for defining tools, not for running Python.) Our intro post explains why that code gets a JavaScript sandbox rather than a Linux machine. That's enough for glue code that calls a tool, filters the result and calls another. An agent that has to run a test suite or an arbitrary CLI needs a Linux sandbox instead.

Agent runtime vs sandbox: your code vs the model's

The test: ask whose code runs there. A runtime hosts trusted code: the harness and its plumbing. A sandbox hosts untrusted code: whatever the model wrote. Our intro post draws the same line. The model's code needs a sandbox; the harness is code you know, so it doesn't.

That gives you two topologies:

1. **Harness inside the sandbox.** Run Claude Code or Codex in a container, and the whole container becomes the sandbox, because the harness and the model's shell commands share one machine. Persisting and waking those containers is a runtime you build. See how to run Claude Code headless in Docker.
2. **Harness outside the sandbox.** The harness runs in the runtime, and only tool execution or model-written code goes into a sandbox. Claude Managed Agents and the OpenAI Agents API work this way, with a vendor-hosted or self-hosted sandbox. So does camelRun, with its JavaScript sandbox.

Anthropic and OpenAI both call that sandbox configuration an **environment**: it

Anthropic and OpenAI both call that sandbox configuration an **environment**. It says where a session runs and executes commands. The phrase "agent runtime environment" can refer to this layer, but vendors use the terms differently. Our guide to building an agent uses "environment" more broadly, for the runtime and sandbox together. Some products span both layers. AgentCore Runtime, for example, is a runtime whose sessions are microVM sandboxes. Judge a product by what it does, not by its name.

Where your prompt and tools fit

The agent's purpose sits on top of the stack: the instructions you write and the tools you give it, which together tell the agent who it is and what its job is, plus the model you pick to carry it out. A harness adds your instructions to its own system prompt and puts your tools next to its built-in ones. A framework leaves all of it to you.

Where tools run matters as much as which tools exist. Your tools are trusted code that carries your credentials and your authorization checks, so they belong in your app or behind your API. Model-written code belongs in the sandbox. If you mix the two up, your credentials end up within reach of code nobody reviewed.

camelRun is a runtime with a built-in harness; per our intro post, the harness is built on Pi. Here is the camelRun quickstart, with the layers marked below it:

```
import { Agents, schema, tool } from "@camelai/run";

const agents = new Agents(); // reads CAMELAI_API_KEY

// A tool is an ordinary function: it runs here, in your process.
const weather = tool({
  description: "Today's weather in a city",
  input: schema.Object({ city: schema.String() }),
  execute: ({ city }) => ({ city, forecast: "sunny", highC: 24 }),
});

// The same key is the same agent, with its history, every time you
const agent = await agents.upsert("quickstart", {
  instructions: "You are a concise assistant.",
  tools: { weather },
});
```

```
const run = await agent.run("Should I bring an umbrella in Lisbon tc
console.log(run.text);

await agents.close();
```

- **Purpose:** `instructions` and `tools`. `weather` runs in your process, with your credentials, not in the runtime or the sandbox.
- **Runtime:** `upsert("quickstart", …)` returns the same agent, with its history, from any process until you delete it.
- **Harness:** `agent.run(...)` hands the turn to the harness, which calls the model, calls `weather` over the SDK's connection and returns the answer.
- **Sandbox:** when the model writes JavaScript to call its tools, that code runs in the sandbox.

"Agent = Model + Harness" leaves out the address

The formula "Agent = Model + Harness" comes from The Anatomy of an Agent Harness, a March 2026 post by LangChain's Vivek Trivedy. The post counts everything that isn't the model as harness. That's a good lens for harness engineering: tuning the prompts, tools, context management and hooks that shape an agent's behavior as much as the model does.

As an architecture diagram, though, the formula hides where the agent lives. A model plus a harness on your laptop is an agent for as long as the process runs. Inside a product, new questions appear:

- Where does each customer's agent live between messages?
- What wakes it when a ticket changes at 3 a.m.?
- What happens to a turn that was running during a deploy?

Read literally, the formula files the sandbox and the hosting service under harness. Yet those pieces fail differently and cost differently, and they often come from different vendors. So we split them out:

> *Agent = model + harness + runtime + a sandbox (if the model runs code) + your prompt and tools.*

Which layers do you need?

- **A script or one-off task:** a harness, run on your machine or in CI. The process itself is runtime enough.
- **A custom workflow with LLM steps:** a framework, plus a durable execution engine if steps run long or must survive restarts.
- **An agent per customer, ticket or thread in your product:** a runtime. Either it has a built-in harness and you write the prompt and tools, or you bring your own harness.
- **An agent that builds, tests or runs arbitrary CLIs:** a Linux sandbox (a microVM or gVisor), with the harness either inside it or driving it from outside.
- **An agent whose code only glues tool calls together:** a light sandbox is enough.

camelRun fits the third and last cases. It's a runtime with a built-in harness and a JavaScript sandbox, and it works with any model. If your agent needs a Linux box, use a harness in a container, Claude Managed Agents, the OpenAI Agents API or AgentCore instead.

FAQ

Is Claude Code an agent or a harness? Both, depending on where you draw the box. The Claude Code product is a complete agent: Anthropic's harness and prompt, running on a Claude model. The Claude Agent SDK exposes that harness so you can add your own prompt and tools. The SDK is a harness, not a runtime: when you embed it in your product, you host it.

Do I need a sandbox if my agent never writes code? If the model can only call tools you wrote, and none of them runs code or commands the model supplies, there's nothing untrusted to isolate. Your tools' own authorization checks do the protecting. Many coding harnesses give the model a shell by default, though, so check the tool list before you skip the sandbox.

Where does MCP fit? MCP is a protocol for exposing tools, not a layer. It's how an agent's tools plug into someone else's harness. One MCP server can serve any harness or runtime that speaks MCP, including Claude Code, Codex and camelRun.

Try camelRun

camelRun is a hosted runtime with a built-in harness. You choose any model, write the instructions and keep your tools in your own code. camelRun runs the loop

the instructions and keep your tools in your own code. camelRun runs the loop, keeps each agent's history and files, runs model-written JavaScript in a sandbox, and wakes agents when there's work. It costs $0.01 per agent-hour of active run time, plus model usage, and sleeping agents cost only storage. The runtime is open source under AGPL-3.0, and you can self-host it with one Docker container and Postgres. See camelRun, or follow the quickstart to get an agent running.

SUGGESTED READING

OCT 5, 2026 · 15 MIN READ

How to Build an AI Agent: 3 Ways, From Scratch to Hosted

TIPS

OCT 8, 2026 · 17 MIN READ

Best AI Agent Runtimes and Hosting Platforms Compared (2026)

PRODUCT

OCT 21, 2026 · 12 MIN READ

OpenAI Agent Builder Shutdown: How to Migrate Your Agents

TIPS

OCT 7, 2026 · 13 MIN READ

AWS Bedrock AgentCore Pricing Explained, and an Alternative

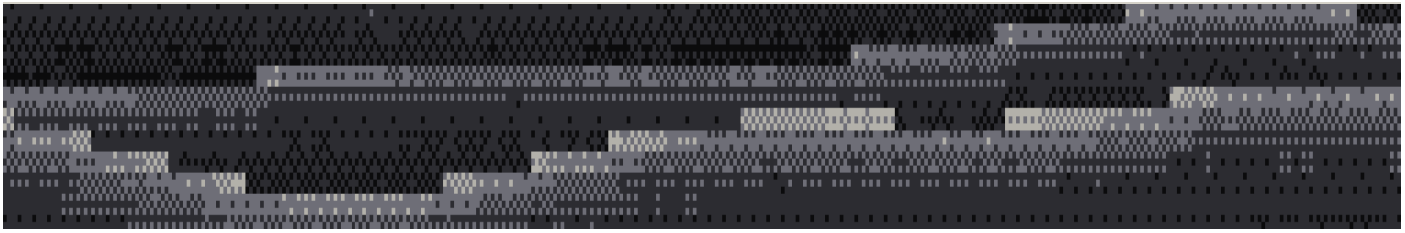
PRODUCT

CAMELRUN · HOSTED AGENT RUNTIME

Run your AI agents on camelRun.

GET AN API KEY

AI AGENT HOSTING WITH CAMELRUN



RUN

BOT

CODE

USE CASES

COMPANY

LEGAL

LEGACY



© 2026 camelAI. All rights reserved.

BACKED BY Y COMBINATOR